Muhammmad Athar Abbas
SP25-BSE-082
Object Oriented Programming
Lab Assignment 2

Task 1:

Animal Class:

```
public class Animal{
   private String name;
   public Animal(String name){
       this.name = name;
   }
   public void speak(){
       System.out.println("This is a generic animal.");
   }
   public String getName(){
       return name;
   }
}
```

Dog class:

```
public class Dog extends Animal{
   public Dog(String name){
       super(name);
   }
   @Override
   public void speak(){
       if(super.getName() == "Buddy"){
           System.out.println("Woof Woof");
```

```
        } else {
            super.speak();
        }
    }
    public static void main(String[] args) {
        Animal a = new Animal("leto");
        a.speak();
        Dog d = new Dog("Buddy");
        d.speak();
        Dog m = new Dog("Max");
        m.speak();
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$ java Dog
This is a generic animal.
Woof Woof
This is a generic animal.
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$
```

Task 2:

Vehicle Class:

```
public class Vehicle{
    private int speed;
    public Vehicle(int speed){
        this.speed = speed;
    }
    public void displaySpeed(){
```

```
        System.out.println("Speed: " + speed);
    }
}
```

Car Class:

```
public class Car extends Vehicle{
    private String brand;
    public Car(String brand , int speed){
        super(speed);
        if(speed > 0){
            this.brand = brand;
        } else {
            this.brand = "Unknown";
        }
    }
    public void displayBrand(){
        System.out.println("Brand: " + brand);

    }

    public static void main(String[] args){
        Car c1 = new Car("Honda" , 0);
        c1.displaySpeed();
        c1.displayBrand();

        Car c2 = new Car("Kia" , 250);
        c2.displaySpeed();
        c2.displayBrand();
    }
```

```
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$ java Car
Speed: 0
Brand: Unknown
Speed: 250
Brand: Kia
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$
```

Task 3:

Shape Class:

```java
public class Shape{
   private String name;
   public Shape(String name){
       this.name = name;
   }
   public void display(){
       System.out.println("Shape: " + name);
   }
   public String getName(){
       return name;
   }
   public static void main(String[] args){
       Circle c = new Circle("Circle");
       c.display();
       c.calculateArea();

       Rectangle r = new Rectangle("Rectangle");
       r.display();
```

```
        r.calculateArea();

        Circle t = new Circle("Triangle");
        t.display();
        t.calculateArea();
    }
}
```

Circle Class:

```
public class Circle extends Shape{

    public Circle(String name){
      super(name);
    }
    public void calculateArea(){
        if(super.getName() ≠ "Circle"){
            System.out.println("Invalid Shape");
        } else {
        System.out.println("Area: " + Math.round(5*5 * Math.PI));
        }
    }
```

Rectangle Class:

```
public class Rectangle extends Shape{
    public Rectangle(String name){
        super(name);
    }
    public void calculateArea(){
        if(super.getName() ≠ "Rectangle"){
```

```
            System.out.println("Invalid Shape");
        } else {
        System.out.println("Area: " + 6*4);
        }
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$ java Shape.java
Shape: Circle
Area: 79
Shape: Rectangle
Area: 24
Shape: Triangle
Invalid Shape
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$
```

Task 4:

Employee Class:

```
public class Employee{
    private String name;
    private int salary;
    public Employee(String name, int salary){
        this.name = name;
        this.salary = salary;
    }
    public void showDetails(){
    System.out.println("Name: " + name + ", Salary: " + salary);
    }
    public int getSalary(){
```

```
        return salary;
    }
    public String getName(){
        return name;
    }
    public static void main(String[] args) {
        Employee e = new Employee("Azhar", 35000);
        e.showDetails();

        Manager m = new Manager("Ajmal" , 86000, "Sales");
        m.showDetails();

        SeniorManager sm = new SeniorManager("Abbas" , 115000,
"Finance" , "Excellent");
        sm.showDetails();

        SeniorManager sm2 = new SeniorManager("Akram" , 90000,
"IT Department", "nill");
        sm2.showDetails();
    }
}
```

Manager Class:

```
public class Manager extends Employee{
    private String department;
    public Manager(String name, int salary, String department){
        super(name, salary);
        this.department = department;
    }
```

```
    @Override
    public void showDetails(){
        System.out.println("Name: " + this.getName() + ", Salary:
" + this.getSalary() + ", Department: " + department);
    }
    public String getDepartment(){
        return department;
    }
}
```

Senior Manager Class:

```
public class SeniorManager extends Manager{
    private String perfomanceRating;
    public SeniorManager(String name, int salary, String
department, String perfomanceRating){
        super(name, salary, department);
        this.perfomanceRating = perfomanceRating;
    }
    @Override
    public void showDetails(){
            System.out.print("Name: " + this.getName() + ",
Salary: " + this.getSalary() + ", Department: " +
this.getDepartment());
        if(this.getSalary() > 100000){
            System.out.println(", Performance Rating: " +
perfomanceRating);
        } else {
        System.out.println(", Perfomance Rating: " + "Not
Available");
```

```
        }
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$ java Employee
Name: Azhar, Salary: 35000
Name: Ajmal, Salary: 86000, Department: Sales
Name: Abbas, Salary: 115000, Department: Finance, Perfomance Rating: Excellent
Name: Akram, Salary: 90000, Department: IT Department, Perfomance Rating: Not Available
[athar@athar ~/semester2/OOP/java_programs/drivelabs/lab7]$
```

Task 5:

Appliance Class:

```java
public class Appliance{
    protected  boolean isOperational = true;
    public Appliance(){};
    public void turnOn(){
        System.out.println("Appliance is turned on.");
    }
    public void turnOff(){
        System.out.println("Appliance is turned off.");
    }
    public void setOperational(boolean isOperational){
        this.isOperational = isOperational;
    }
    public static void main(String[] args){
        Fan f1 = new Fan(false, false);
        f1.turnOn();
        f1.turnOff();
```

```
        Light l1 = new Light(false,75);
        l1.turnOn();
        l1.turnOff();

        Fan f2= new Fan(true, true);
        f2.turnOn();
        f2.turnOff();

        Light l2 = new Light(true, 0);
        l2.turnOn();
        l2.turnOff();
    }
}
```

Fan Class:

```
public class Fan extends Appliance{
    private boolean isSummer;
    private boolean isBroken;
    public Fan(boolean isSummer, boolean isBroken){
        this.isSummer = isSummer;
        this.isBroken = isBroken;
        if(isBroken){
            isOperational = false;
        }
    }
    @Override
    public void turnOn(){
        if(isSummer && isOperational){
```

```
            System.out.println("Fan is turned on.");
        } else if(!isSummer && isOperational){
            System.out.println("Fan cannot be turned on in
winter");
        } else {
            System.out.println("Fan is broken and cannot be
turned on.");
        }
    }
    @Override
    public void turnOff(){
        System.out.println("Fan is turned off.");
    }

}
```

Light Class:

```
public class Light extends Appliance{
    private boolean isDaytime;
    private int powerLevel;
    public Light(boolean  isDaytime, int powerLevel){
        this.isDaytime = isDaytime;
        this.powerLevel = powerLevel;
        if(powerLevel ⩽ 0){
            isOperational = false;
        }
    }
    @Override
    public void turnOn(){
```

```
        if(isOperational && !isDaytime){
            System.out.println("Light is turned on at " +
powerLevel + "% brightness.");
        } else if(isOperational && isDaytime){
            System.out.println("Light cannnot be turned on during
the day.");
        } else {
            System.out.println("Light is broken and cannot be
turned on.");
        }
    }
    @Override
    public void turnOff(){
        System.out.println("Light is turned off.");
    }
}
```

Output: